On CRUD in Haskell

2021-12-15

即使用上了优雅的 Haskell，我做的仍然是 CRUD 这种搬砖活。然而不幸的是，即使是 Haskell 强大的类型系统，也不能避免开发中产生的许多错误。比如说，就在动笔之前，我还刚刚修了个 bug——在 Postgres 中，多维数组的子数组不能用有不同的长度，因此，在使用 `array_agg` 将数组聚合起来的时候，是极有可能报错的——而再强大的 SQL eDSL 都不能在编译期发现这样的错误。算起来，这已经是我第三次被这个 bug 坑害了。虽然如此，Haskell 还是写得我心情愉快，所以还是忍不住想写这篇文章。

1 项目架构

这里所谓的"项目架构"是个狭义概念。谈到 API server，那肯定是由 Model 和 Controller 两大部分构成。其中，Model 的实质是在造/解释一个 eDSL，而 Controller 是在使用这个 eDSL。那么我们就有两种选择构造eDSL：Free Functor 和 Tagless Final。本来我是想用 free 的，但是这个库实在是太简陋，不适合工程，而 polysemy 的 example 里面充斥着各种类型和不知道功能的 Template Haskell，我甚至都没有看明白… 而众所周知，由于 n^2 实例的原因， mtl 对于自定义新的 Monad Transformer 非常不友好。最后我选择了 fused-effects 这个号称拥有 Algebraic Effects 的库。

直接使用上，除了有些时候需要手动注明类型之外，fused-effects 的使用体验和 mtl 类似；而当你想要手动定义新的 effect（相当于 Monad Transformer）时，除了需要写一些 boilerplate 之外，只需要线性代码量的体验还是极好的。fused-effects 将 Effect 和 Carrier 分开来了——Effect 描述效果，Carrier 描述解释一个效果的方式，用一个 instance 表明 Carrier 能解释对应 Effect。fused-effects 还允许一个 Effect 拥有多个 Carrier，比如 Trace（打 log）这个 Effect 就有 "忽略输出" 和 "输出到 stdout" 等多种 Carrier。由于听祖与占上仙说粗粒度的效果有违 Haskell 强类型的原则，我把数据库中的每张表封装成一个 Effect（比如 `DBUser`，`DBComment`），使用对应效果时，可以分开来写：`Has DBUser sig m => m a`。（话说这个 `Has` 很帅有木有？）

另外，对于 Effect 里的每一个值构造子，通常都要手写一个 `send` 的版本；可以写一个 Template Haskell 函数，一次性 `send` 所有的值构造子。

另外，虽然我不怎么关心这里的性能问题，但是在 fused-effects 的 README 里面有一个 Benchmark，其中唯二比 fused-effects 快的库是 mtl 和 eff，而后者甚至还未完工，并且最近更新还在去年… emm…

2 服务器框架

至少在我有限的见识里，Haskell 中所有的服务器框架都是基于 WAI / Warp 的（经上仙指点，snap 不是），在此之上进入我的考虑范围的有 yesod 和 servant （其实是别的没怎么听人说过）。然而算是有些主观原因吧，在我的英语和 Haskell 都不好的时候去看 yesod 的文档，把我看了个云里雾里，对 yesod 留下了不好的印象。然而公正的说，yesod 由于历史局限性，搞了一套莎翁全家桶出来，然而现在已经是前后端分离的时代了。相比之下，servant 则是专门的 API server 框架，于是我几乎毫不犹豫地选择了后者。Snoyman 大失败！

实际使用起来，首先一个字，酷。虽然我不知道实现原理（感觉 Haskell 里 type 的 quantity 应该都是 0，并且 servant 似乎也没有用什么 Template Haskell 在编译时取类型信息），但是 type-level 的路由实在是非常炫酷，type safety 自然也是杠杠的（对比一下，go 的 echo 对请求体要手动 Bind）。 其次，servant 对各种 Authentication 的支持也比较好，并且还能够和 type-level API 很好地集成。然后，servant 能够很方便地和上面的 fused-effects 库一起使用。Servant 有个 `hoistServer`，接收一个 Natural Transformation 作参数，这个参数刚好就是 fused-effects 中各个 Carrier 的 runner。

优点谈完了谈缺点：

1. servant 没有自定义返回的错误类型的能力。比如说，如果我想让我的错误返回值是 JSON 格式：`{"status": 500, "error": "Something went wrong."}`，那 servant 是做不到的。虽然有个库 servant-errors 可以做这个事情，然而如果我想要在 Debug 模式下多输出一个`errHint`字段，那唯一的解决办法恐怕只有把`(error, errHint)` serialize 成`ByteString`放在`ServerError`里，再自己写个中间件来格式化错误信息了吧。最后我还是放弃了`errHint`字段。
2. 然后这个其实不是 servant 的问题，是 WAI/Warp 的问题。在 WAI 中，Response 的底层表示是一个流，也就是说，当 WAI 拿到一个 Response 的时候，它是没有被完全求值的。一方面，这是 Haskell 惰性求值优越性的体现；但是另一方面，我们也无法预先得知这个流之中是否有 bottom。当然，可以写一个中间件把整个流 `deepseq` 一遍，但是我在中间件里同样不能知道这个 Response 是只有 100 个字符，还是长达 1GB，将其求值到 Normal Form 需要占用我多少内存。这在别的语言中是不会出现的问题：异常不会在求值时产生。于是，可能在别的语言中，能够正确报出 500 server error；但是在 Haskell 中，我却无法抓到 Response 里的 error。
3. 配 custom ErrorFormatters 有点麻烦（？）

总的来说，servant 还是相当好用的，至少大部分的地方还是保持了相当的简洁，尤其是 API type 和 function type 的一一对应，实在是让人写得非常舒爽。最高にハイってやつだ！

3 数据库框架

在选择数据库之前，我首先参考了 这篇文章 。由于我对各种抽象不但没有抗拒心理，反而十分欢迎（否则我写 Haskell 干什么），因此我首先是毫不犹豫地选择了 Opaleye 。然而不久之后，我却放弃了已经写了一般的 Model 部分，转而改用了 beam 。

实话实说，Opaleye 确实是相当优秀的数据库框架，它几乎做到了一个轻量级框架的极致。如果不是非常复杂的数据库业务，我是十分推荐 Opaleye 的。Opaleye 将每张表实现为`ProductProfunctor`的实例，这就奠定了它强类型的基础——即使是 aggregate，Opaleye 也用 `ProductProfunctor` 进行了优雅的实现，相较而言，beam 的 aggregate 就相当丑陋了，无怪乎 Opaleye 在文档中说 "Type safe aggregation is the jewel in the crown of Opaleye." 我还专门为 Opaleye 写了一套 TH，拿来生成一些 boilerplate。不过我在 Opaleye 写双层 aggregate 的时候，总是通不过类型检查，也不知道是我自己还是 Opaleye 的问题。

到此为止 Opaleye 都可以说是出众的，然而越过了这个类型安全的极限，就可以感到 Opaleye 在摆烂。Opaleye 不能 insert selected result，没有 on conflict update，更关键的是这些缺失的功能还难以自己实现。无奈我只好转向 beam。不过 Opaleye 在0.8版本把原本的`DoNothing`构造器隐藏了，可能是准备开始做 on conflict update的支持了，对此我表示十分期待。

相较于 Opaleye，beam 唯二的好处就是功能全面，以及生成的 SQL 比较简单，而后者几乎可以忽略不计。beam 几乎可以说是最重的数据库框架，比 Opaleye 是高出两个重量级，凡是一切 SQL 有的功能，它几乎都用类型描述了一遍；而即使是缺失的功能，也能方便地用 `customExpr_` 自己加上。但是缺点可以说是只多不少：

- beam 是反对 Template Haskell，支持 `Generic` 的，而刚好我又 favour Template Haskell more than `Generic`，觉得 `Generic` 会拖慢运行速度。
- beam 对 Postgres 的 `array_agg` 很不友好。假设有 `data SomeT f = ... deriving (Generic, Beamable)`，然后有一个 `AnotherT f` 包含 `SomeT f` 的一个 `Vector`（或者 `List`），那 `AnotherT f` 是无法 `derive` 为 `Beamable` 的实例的。
- 不像 Opaleye 能直接用表的结构来做 aggregate，beam 的 aggregate 只能使用元组，这就导致 beam 的 aggreagte 可以极其丑陋，实在不应该是一门强类型语言所为，并且 aggregate 完成之后，还得再用丑陋的 zipWithN 处理一下结果（定义一个 `ZipVector` 可以部分解决这个问题）。并且，beam 只支持长度不超过 8 的元组，然而稍微复杂的 aggregate，涉及的字段数都可以随便超过 8。（对比一下，Opaleye 支持长达 62 的元组！）更要命的是，如果你想手动为更长的元组定义实例，你会发现定义所用到的一个方法是被隐藏起来的！（虽然我还是用了 这里 的方法成功 hack 出了这个方法。）
- 虽然 Opaleye 是完全没有 migration，可是 beam 的 migration 表现同样不怎么样。虽然不排除是我不会用的原因，但是 beam migration 的能力似乎仅限于从一个空的数据库中创建表，且不论自动增删字段，就是数据库中少了一个表，beam 都不能自己加上。并且，beam 的 migration 没有对外键的支持（虽然可以根据 这里 的方法手动加上）。我本来还想用用 beam-automigrate 的，可是这玩意居然还在依赖 aeson < 1.5，果断放弃。

总的来说，虽然我自己是在用 beam，但我的建议是能用 Opaleye 就尽量用 Opaleye。并且，我十分期待 Opaleye 今后的能够补上欠缺的功能。只有当 Opaleye 力所不能及的时候，我才建议使用 beam。

说到这里就顺便提一提 persistent + esqueleto 吧。作为我同样了解过的数据库框架，我的评价是 beam 都不如。首先是类型定义都得写在 TH 里面，未免过于繁琐；其次是在 persistent 里面可以写出没有 on 的 join 来，类型安全没有保障；最后，功能没有 beam 全面，语法还没有 beam 简洁（至少我主观上觉得如此）。Snoyman 大失败 × 2！（虽然我还是爱用他的 stack 就是了。）

4 杂项

顺便讲讲开发遇到的一些小问题的解决吧。

Servant 的 basic auth 是和类型绑定的。这就是说，每一种 basic auth scheme 只能对应一种结果类型。如果想让两种 basic auth scheme 得到相同的结果类型的话，可以用 phantom type。

Aeson 序列化的方式同样是和类型绑定的，一种类型只能有一个 `ToJSON/FromJSON` 实例。如果想要从 `ToJSON` 的结果中除去某个字段，可以定义 `newtype`；如果想在 `ToJSON` 的结果中加上某个字段，可以定义一个类型乘法，并且手写这个乘法类型的 `ToJSON` 实例。另外，aeson 的 TH 不能为 type synonym 生成实例，但是 `Generic` 可以。

如果让代码在 docker 容器里编译，会导致一旦依赖改变，所有的依赖都要重新编译。不如先在本机编译好，再把二进制文件拷到 docker 镜像中（逃）。